CSE274:Applied Machine Learning

UNIT 1

UNIT I Syllabus Contents

- Data Pre-processing : Types of data, Data pre-processing, data leakage concept, handling missing values, outliers handling,, handling categorical data, scaling and normalization, class imbalance handling

1. The "Big Four" Libraries

Library	Alias	The Job	Real-world Analogy
NumPy	np	Math & Arrays	The **Calculator**. It handles complex numbers and matrices very fast.
Pandas	pd	Data Tables	The **Excel Spreadsheet**. It organizes data into rows and columns (called DataFrames)
Matplotlib / Seaborn	plt / sns	Visualization	The **Sketchbook**. It turns numbers into charts and graphs.
Scikit-Learn	Sklearn	The ML Brain	The **Workroom**. It contains all the ready-made algorithms (linear regression, trees, etc.).

2. What do these look like in action?

Let's look at a simple example. We want to predict if a plant is "Tall" based on its "Water Intake."

```
# --- THE IMPORTS (Setting up our tools) ---
import numpy as np        # For math
import pandas as pd       # For data tables
from sklearn.linear_model import LinearRegression # The specific 'Brain' we want

# --- STEP 1: Creating Data (Pandas & NumPy) ---
# We use NumPy to create lists of numbers
# We use Pandas to put them in a table
data = {
    'Water_Liters': np.array([1, 2, 3, 4, 5]), # Input (X)
    'Plant_Height': np.array([10, 20, 30, 40, 50]) # Target (y)
}
df = pd.DataFrame(data)

print("--- Our Dataset (Pandas DataFrame) ---")
print(df)

# --- STEP 2: Building the Model (Scikit-Learn) ---
# 1. Initialize the 'Brain'
model = LinearRegression()
```

```
# 2. Fit the model (Training)
# .values.reshape(-1, 1) is just a NumPy trick to make the data 2D
X = df[['Water_Liters']]
y = df['Plant_Height']
model.fit(X, y)

# 3. Predict (The AI part)
new_plant_water = [[6]] # A plant gets 6 liters
prediction = model.predict(new_plant_water)

print(f"\nPrediction: A plant with 6L of water will be {prediction[0]}cm tall.")

"""
EXPLANATION :
- Pandas (df) held our data safely.
- NumPy (np) handled the array math behind the scenes.
- Scikit-Learn (model) looked at the data and learned the rule:
  'Height = Water * 10'.
"""
```

3. Key Vocabulary you will see often:

- **DataFrame:** A table in Pandas (like an Excel sheet).
- **Series:** A single column from that table.
- **.fit():** This is the command to **Train** the model. It's the "Learning" phase.
- **.predict():** This is asking the model for its **Opinion** on new data.
- **.transform():** This is used in preprocessing (like Scaling) to **Change** the data.
- **X (Capital):** Usually represents your **Features** (Inputs like Age, Height).
- **y (Lowercase):** Usually represents your **Target** (The Answer you want to predict).

4. Why use these instead of normal Python lists?

- If you have 1 million rows:
- Normal Python lists are **slow** (it processes one item at a time).
- **NumPy and Pandas** use "Vectorization." They perform the math on the whole column at once using your computer's hardware efficiently. It can be 100x faster

Types of Data

1. Qualitative (Categorical) Data

Categorical data represents characteristics. You cannot perform meaningful math (like finding an average) on these directly.

A. Nominal Data (Latin for "Name")

- These are labels without any intrinsic ordering or ranking.
- **Examples:** Marital status (Single, Married), Nationality, Blood Type, or even "ID Numbers" (since the number doesn't represent a quantity).
- **Mathematical Constraint:** You can only calculate **Mode** (the most frequent) and **Frequency**.

B. Ordinal Data (Order Matters)

- These labels have a logical rank, but the "distance" between the ranks isn't mathematically defined.
- **Examples:** Shirt sizes (S, M, L, XL), Customer Satisfaction (1-5 stars), or Seniority (Junior, Mid, Senior).
- **Mathematical Constraint:** You can calculate **Mode** and **Median**, but a "Mean" is technically incorrect because the jump from "Small" to "Medium" might not be the same as "Medium" to "Large."

2. Quantitative (Numerical) Data

Numerical data represents measurable quantities.

A. Discrete Data

- Countable values that cannot be split into smaller parts (usually integers).
- **Examples:** Number of pets in a house, number of clicks on an ad. (You can't have 2.5 dogs).

B. Continuous Data

- Values that can be measured on an infinite scale and can take any decimal value.
- **Examples:** Temperature ($22.51°C$) ,(Salary ($50{,}000.75$)), or Height.
- **Mathematical Constraint:** You can calculate **Mean, Median, Mode, Standard Deviation, etc.**

Python Implementation: Identifying Data Types

In a real project, your first step is loading the data and checking how Python "sees" it.

```
import pandas as pd
# Creating a sample dataset
data = {
    'Employee_ID': [101, 102, 103, 104],        # Nominal (treated as int)
    'Department': ['HR', 'Tech', 'Tech', 'Ops'], # Nominal
    'Seniority': ['Junior', 'Senior', 'Mid', 'Junior'], # Ordinal
    'Children_Count': [0, 2, 1, 3],             # Discrete
    'Salary': [45000.50, 80000.00, 60000.25, 42000.00] # Continuous
}
df = pd.DataFrame(data)
```

```
# 1. Check Data Types
print("--- Data Types ---")
print(df.dtypes)

# 2. Basic Statistical Summary
# By default, describe() only shows numerical columns
print("\n--- Numerical Summary ---")
print(df.describe())

# To see categorical summary
print("\n--- Categorical Summary ---")
print(df.describe(include=['object’]))
```

- **Why does this matter for Python?**
- **object or category:** This is how Pandas stores Qualitative data.
- **int64 or float64:** This is how Pandas stores Quantitative data.

Deep-Dive Challenge:

- Look at the Seniority column in the code above. Even though it is a string (Junior, Mid, Senior), we know it has an order.

To make a computer understand that

- **Senior > Mid > Junior**, we use **Ordinal Encoding**. If we leave them as strings, the model treats them as random labels. If we use standard alphabetical sorting, it might think "Junior" (J) comes before "Senior" (S) just because of the letter, which is wrong.

Here is how you handle this in Python using **Pandas** and **Scikit-Learn**

Method 1: Using Pandas map() (Most Intuitive).

This is the best way when you have a small, specific set of ranks. You manually define the "weight" of each label.

```
import pandas as pd
# Our sample data
df = pd.DataFrame({'Seniority': ['Junior', 'Senior', 'Mid', 'Junior']})

# 1. Define the logical order in a dictionary
ranking_dict = {
    'Junior': 1,
    'Mid': 2,
    'Senior': 3
}

# 2. Map the strings to the numbers
df['Seniority_Encoded'] = df['Seniority'].map(ranking_dict)

print(df)
```

Method 2: Using Scikit Learn OrdinalEncoder (Best for Pipelines)

In professional Machine Learning, we use the Scikit-Learn OrdinalEncoder. This is safer because it can be integrated into a Pipeline to prevent **data leakage.**

```
from sklearn.preprocessing import OrdinalEncoder

# Sample data
data = [['Junior'], ['Mid'], ['Senior'], ['Junior']]
df_sklearn = pd.DataFrame(data, columns=['Seniority'])

# 1. Define the order (from lowest to highest)
categories_order = ['Junior', 'Mid', 'Senior']

# 2. Initialize the encoder with the specific order
encoder = OrdinalEncoder(categories=[categories_order])

# 3. Fit and transform the data
df_sklearn['Seniority_Encoded'] = encoder.fit_transform(df_sklearn[['Seniority']])

print(df_sklearn)
```

Why we do this (The "In-Depth" Reason)

Most ML algorithms (like Linear Regression or Support Vector Machines) use math like **y=mx+b**

If "Seniority" is a string, the math breaks: $y=m*("Senior)+b$ is impossible.

By encoding "Junior" as 1 and "Senior" as 3, the model can now calculate that "Senior" has **3x the weight** of a "Junior" in the prediction.

Quick Knowledge Check:

- If you had a column called **"City"** (New York, Mumbai, London), would you use this same "1, 2, 3" encoding?

Nominal vs. Ordinal Encoding

- **Ordinal Encoding:** Use only when there is a clear rank (e.g., Cold < Warm < Hot).
- **One-Hot Encoding:** Use for Nominal data (e.g., Cities, Colors). It creates new columns for each category with 1s and 0s.

Python Deep-Dive: Handling Nominal Data

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

# Create a sample dataset with Nominal data
df = pd.DataFrame({'City': ['New York', 'Mumbai', 'London', 'Mumbai']})

# 1. Initialize the OneHotEncoder
# sparse_output=False gives us a readable array instead of a matrix
encoder = OneHotEncoder(sparse_output=False)

# 2. Fit and Transform the data
# We use .fit_transform() to learn the categories and convert them at once
city_encoded = encoder.fit_transform(df[['City']])

# 3. Create a new DataFrame with the encoded columns
# encoder.get_feature_names_out() retrieves the original category names for headers
encoded_df = pd.DataFrame(city_encoded, columns=encoder.get_feature_names_out(['City']))
```

```
# 4. Concatenate with original data to see the result
result = pd.concat([df, encoded_df], axis=1)

print("--- One-Hot Encoded Cities ---")
print(result)
```

Pro-Tip: The "Dummy Variable Trap"

- When you use One-Hot Encoding, if you have 3 cities, you actually only need 2 columns to represent them. If a row is 0 for London and 0 for Mumbai, it **must** be New York.
- Including all 3 columns can cause **Multicollinearity** (where variables are perfectly predictable from others). In Scikit-Learn, you can prevent this by setting drop='first' in the encoder.

Summary of Types vs. Encoding

Data Type	Example	Best Encoding
Nominal	City, Gender	One-Hot Encoding
Ordinal	Rating (1-5), Size	Ordinal Encoding
Discrete	# of Children	Leave as is (it's already a number)
Continuous	Temperature, Price	Leave as is (but consider **Scaling** later)

The "Dummy Variable Trap" Explained

- **The Trap:** Including all columns created by One-Hot Encoding leads to redundant data.
- **The Math:** N categories → $N - 1$ columns.

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

# --- STEP 1: Create simple data ---
# We have 3 types of weather.
data = {'Weather': ['Sunny', 'Rainy', 'Cloudy', 'Sunny']}
df = pd.DataFrame(data)

# --- STEP 2: Initialize Encoder with drop='first' ---
# 'drop=first' tells Python: "Delete the first category column it creates"
# This avoids the Dummy Variable Trap.
encoder = OneHotEncoder(sparse_output=False, drop='first')
```

```
# --- STEP 3: Transform the data ---
encoded_data = encoder.fit_transform(df[['Weather']])

# --- STEP 4: Put it back into a nice Table (DataFrame) ---
# Note: Since we dropped one, we only get 2 columns back for 3 categories.
encoded_df = pd.DataFrame(
    encoded_data,
    columns=encoder.get_feature_names_out(['Weather'])
)

print("--- Data with 'drop=first' ---")
print(encoded_df)
```

UNDERSTANDING THE OUTPUT:

1. We had 3 categories: Cloudy, Rainy, Sunny.
2. The encoder dropped 'Cloudy' (the first one alphabetically).
3. We are left with: 'Weather_Rainy' and 'Weather_Sunny'.

HOW TO READ THIS:

- If Rainy=0 and Sunny=0, the computer knows: "It must be Cloudy!"
- If Rainy=1 and Sunny=0, the computer knows: "It is Rainy!"
- If Rainy=0 and Sunny=1, the computer knows: "It is Sunny!"

"Why did my model get 99% accuracy in the classroom but 50% accuracy in the real world?"

- **Data Leakage**. This is perhaps the most important concept to master because it defines whether your model is actually "smart" or just "memorising the answers."
- **The Result:** You get a model that looks perfect during training (99% accuracy) but fails completely when used on real, new data.

Two Main Types of Leakage

1. Preprocessing Leakage (Train-Test Contamination)

This happens when you calculate statistics (like the **Mean**, **Max**, or **Category Frequencies**) using the *entire* dataset before splitting it.

Example: You want to fill missing heights with the "Average Height." If you calculate the average using the whole dataset, your training data now knows the average of the test data. The "future" has leaked into the "past."

2. Target Leakage

This happens when your input features include data that wouldn't actually be available at the time of prediction.

Example: You are building a model to predict if a customer **will get a loan**. If your data includes a column for "Monthly Installment Amount," that is leakage! You only have an installment amount *after* the loan is approved. You are using the result to predict the cause.

How to Prevent It: The Golden Workflow

The only way to be safe is to follow this sequence strictly:

- **Split** your data into Train and Test sets first.
- **Fit** your preprocessors (Encoders, Scalers, Imputers) **only** on the Training set.
- **Transform** both the Train and Test sets using what you learned from the Train set.

Python Code: The Right vs. Wrong Way

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

# --- STEP 1: Create dummy data (e.g., Exam Scores) ---
data = {'Score': [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]}
df = pd.DataFrame(data)

# --- THE WRONG WAY (Leakage) ---
# Calculating the 'Max' score of the WHOLE dataset before splitting.
bad_scaler = MinMaxScaler()
df['Bad_Scaled'] = bad_scaler.fit_transform(df[['Score']])
# Now, the training part of the data knows that 100 is the absolute max of the test set.
```

```
# --- THE RIGHT WAY (No Leakage) ---
# 1. Split first
train, test = train_test_split(df[['Score']], test_size=0.2,
random_state=42)

# 2. Initialize Scaler
good_scaler = MinMaxScaler()

# 3. FIT ONLY ON TRAIN (Learn that the max is, say, 80)
good_scaler.fit(train)

# 4. TRANSFORM BOTH (Apply that '80' max to the test set)
train_scaled = good_scaler.transform(train)
test_scaled = good_scaler.transform(test)
```

Pro-Tip for Later: Scikit-Learn Pipelines

In professional code, we use Scikit-Learn Pipelines.

A Pipeline "wraps" your preprocessing and model together, so when you call .fit(), it automatically ensures no data from the test set leaks into the training process.

Handling Missing Values (Imputation)

This is where we actually start fixing broken data!

Moving into **Handling Missing Values**. In the real world, data is messy—sensors fail, people skip survey questions, and systems glitch. You will rarely find a perfect dataset.

There are three main "reasons" why data goes missing, and knowing which one you're dealing with helps you choose the right fix.

1. The Three Types of Missingness

MCAR (Missing Completely At Random): The missing data is just a random accident. (e.g., A wind gust knocked over a sensor for 5 minutes).

MAR (Missing At Random): The missingness is related to *other* observed data. (e.g., Men are less likely to disclose their weight in a survey, but we have their "Gender" column).

MNAR (Missing Not At Random): The missingness is related to the value itself. (e.g., People with very high debts don't want to answer the "Total Debt" question).

2. Common Strategies to Fix Missing Data

1. Deletion: Delete the row or the whole column. (Risky if you lose too much info).

2. Imputation (Filling): Replacing the "NaN" (Not a Number) with a specific value.

- **Numerical:** Fill with **Mean** (Average), **Median** (Middle value), or **Mode** (Most frequent).
- **Categorical:** Fill with the **Mode** or a new label like **"Missing"**.

3. Python Code: Handling Missing Values

```
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer

# --- STEP 1: Create a messy dataset ---
# np.nan is how Python represents a "Missing Value"
data = {
    'Age': [25, 30, np.nan, 40, 100],  # 100 is an outlier!
    'City': ['London', 'Paris', 'London', np.nan, 'Paris']
}
df = pd.DataFrame(data)

# --- STEP 2: Simple Pandas Approach (Quick Check) ---
# Check how many values are missing in each column
print("Missing Values Count:\n", df.isnull().sum())
```

```
# --- STEP 3: Professional Approach using SimpleImputer ---
# A. For Numerical (Age):
# We use 'Median' because 'Mean' is easily ruined by outliers like the age '100'.
num_imputer = SimpleImputer(strategy='median')
df['Age'] = num_imputer.fit_transform(df[['Age']])

# B. For Categorical (City):
# We use 'most_frequent' (Mode) to fill the gap with the most common city.
cat_imputer = SimpleImputer(strategy='most_frequent')
df['City'] = cat_imputer.fit_transform(df[['City']])

print("\n--- Fixed Data ---")
print(df)

"""
EXPLANATION:
1. 'isnull().sum()' is your detective tool. It tells you exactly where the holes are.
2. We chose MEDIAN for Age because the average (Mean) of [25, 30, 40, 100] would be
   roughly 48. But most people are 25-40! The Median (30) is a much 'safer' guess.
3. We used 'most_frequent' for City because you can't calculate an 'average city'.
"""
```

Pro-Tip: The "Missingness" Indicator

- Sometimes, the fact that a value is missing is **valuable information** itself. For example, if "Credit Score" is missing, it might mean the person has never applied for a loan.
- In Scikit-Learn, you can set add_indicator=True. This creates an extra column that tells the model: *"Hey, this value was originally missing!"* This often improves model accuracy.

Knowledge Check:

- If you have a column for **"Annual Income"** and it has many missing values, but you also have an **"Outlier"** (one person earns $10,000,000 while others earn $50,000), would you fill the missing values with the **Mean** or the **Median**? Why?

Now, let's move to the culprits themselves: **Outliers Handling**.

1. What are Outliers?

Outliers are data points that differ significantly from other observations. They can be caused by:

- **Measurement errors** (e.g., a weight of 500kg for a human).
- **Natural variance** (e.g., Bill Gates' income in a neighborhood survey).

2. How to Detect Outliers

The most common statistical method is the **IQR (Interquartile Range) Method**:

- $Q1$ The 25th percentile (bottom 25%).
- $Q3$ The 75th percentile (top 25%).
- $IQR = Q3 - Q1$.

Boundaries: Anything below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ is an outlier.

3. Python Code: Detecting and Capping Outliers

- Instead of just deleting outliers (which loses data), we often **"Cap"** them (also called **Winsorization**). This means we bring the extreme values down to the maximum "allowed" boundary.

```
import pandas as pd
import numpy as np

# --- STEP 1: Create data with a massive outlier ---
df = pd.DataFrame({'Income': [45, 50, 52, 48, 47, 500]}) # 500 is the outlier

# --- STEP 2: Calculate IQR ---
Q1 = df['Income'].quantile(0.25) # The value at 25% mark
Q3 = df['Income'].quantile(0.75) # The value at 75% mark
IQR = Q3 - Q1

# --- STEP 3: Define Upper and Lower Bounds ---
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print(f"Any income above {upper_bound} is an outlier.")
```

```
# --- STEP 4: Handle Outliers (Capping) ---
# np.clip limits the values: anything > upper_bound becomes the upper_bound
df['Income_Capped'] = df['Income'].clip(lower=lower_bound, upper=upper_bound)

print("\n--- Data after Capping ---")
print(df)

"""
EXPLANATION:
1. We calculated that '500' is way outside the normal range.
2. Instead of deleting the person earning 500, we 'capped' them at 56.5.
3. This keeps the 'rank' of the data the same (they are still the highest earner)
   but prevents them from ruining the math of models like Linear Regression.
"""
```

4. Other Handling Techniques

- **Trimming:** Removing the outlier rows entirely. Use this ONLY if you are sure the outlier is a mistake/error.
- **Log Transformation:** Applying np.log() to a column. This "squashes" large values closer to the small values, making the distribution look more "normal."
- **Log Transformation** is like a "magic lens" for data. It is particularly useful when your data is **right-skewed** (meaning you have a few very large values that stretch the graph to the right, like Income or House Prices).

Why use Log Transformation?

- **Reduces Variance:** It pulls extreme outliers closer to the mean.
- **Normalizes Distribution:** Many ML models (like Linear Regression) assume your data follows a "Bell Curve" (Normal Distribution). Log transforms often turn skewed data into a bell curve.
- **Linearizes Relationships:** It can turn exponential growth into a straight line, which is much easier for models to learn.

Python Code: Log Transformation

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# --- STEP 1: Create skewed data (House Prices) ---
# Most houses are 100k-500k, but one is 10 million!
data = {'Price': [120000, 150000, 200000, 250000, 300000, 10000000]}
df = pd.DataFrame(data)

# --- STEP 2: Apply Log Transformation ---
# np.log1p calculates log(1 + x) for every value in the column
df['Log_Price'] = np.log1p(df['Price'])

print("--- Original vs Log Transformed ---")
print(df)
```

UNDERSTANDING THE OUTPUT:

1. Look at the 'Price' column: 10,000,000 is 50 times larger than 200,000.

2. Look at the 'Log_Price' column: 16.11 is only about 1.3 times larger than 12.20.

3. The 'distance' between the luxury mansion and the normal house has been mathematically 'shrunk' so the mansion doesn't overwhelm the model.

```
# --- STEP 3: Visualizing the 'Squash' effect ---
# (Optional visualization to see the difference)
print(f"\nOriginal Max/Min Ratio: {df['Price'].max() /
df['Price'].min():.2f}")
print(f"Log-Transformed Max/Min Ratio: {df['Log_Price'].max() /
```

When NOT to use Log Transformation:

- If your data contains **negative numbers** (Log of a negative is impossible).
- If your data is already **normally distributed** (it might make it worse).

Handling Categorical Data

One-Hot Encoding (for Nominal) and **Ordinal Encoding** (for Ordinal).

There is one more common technique: **Label Encoding**.

Label Encoding is often confused with Ordinal Encoding.

Ordinal Encoding: You define the order (Small=1, Medium=2).

Label Encoding: Python assigns a number to each category based on alphabetical order (Apple=0, Banana=1, Cherry=2).

Crucial Rule: Label Encoding should **only** be used for the **Target Variable (y)** (the thing you are trying to predict), never for the **Input Features (X)**.

Python Code: Label Encoding the Target

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# --- STEP 1: Create a dataset where we predict 'Subscription_Type' ---
data = {
    'User_Age': [25, 45, 33, 22],
    'Subscription': ['Premium', 'Basic', 'Free', 'Premium'] # This is our Target (y)
}
df = pd.DataFrame(data)

# --- STEP 2: Initialize LabelEncoder ---
# LabelEncoder is designed specifically for target labels (y).
label_enc = LabelEncoder()

# --- STEP 3: Fit and Transform the Target ---
# It will assign: Basic=0, Free=1, Premium=2 (alphabetical order)
df['Subscription_Encoded'] = label_enc.fit_transform(df['Subscription'])
```

```
print("--- Encoded Target Variable ---")
print(df[['Subscription', 'Subscription_Encoded']])

# --- STEP 4: How to get the original names back ---
# Useful for the end of your project when you want to show real names.
original_names = label_enc.inverse_transform([0, 1, 2])
print(f"\nClasses learned by encoder: {original_names}")

"""
UNDERSTANDING THE CODE:
1. LabelEncoder converts each unique string into a unique integer.
2. We only apply this to 'y' (Subscription).
3. If 'Subscription' was an input feature (X), we would use One-Hot Encoding instead
   to avoid the model thinking Premium (2) is 'greater than' Basic (0).
"""
```

Scaling and Normalization

- Now that our data is numeric and the target is ready, we face a new problem: **Magnitude.**
- Imagine a dataset with:
- **Age:** Range 0 to 100.
- **Annual Income:** Range $20,000 to $500,000.
- If you feed these into a model like K-Nearest Neighbors (KNN), the model will think **Income is 5,000 times more important** than Age simply because the numbers are larger.

To fix this, we use:

- **Normalization (Min-Max Scaling):** Squeezes all values between **0 and 1**.
- **Standardization (Z-score Scaling):** Centers data around a **mean of 0** with a **standard deviation of 1**.
- Most Machine Learning models calculate the distance between points or use gradients (slopes) to find the best answer.
- If one feature has a range of $100{,}000$ and another has a range of 10, the "slope" for the larger number will be so steep that the model will ignore the smaller number entirely.

1. Normalization (Min-Max Scaling)

- Normalization rescales the data into a fixed range, usually **0 to 1**.
- **The Formula:**

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- **When to use it:** When you know the distribution of your data does **not** follow a Bell Curve (Gaussian).
- When you are using algorithms like **Neural Networks** or **K-Nearest Neighbors**.
- **The Downside:** It is very sensitive to outliers. If you have one person earning $10M, everyone else will be squashed to 0.0001

2. Standardization (Z-Score Scaling)

- Standardization centers the data around 0 and makes the "spread" (standard deviation) equal to 1.
- **The Formula:** $$X_{std} = \frac{X - \mu}{\sigma}$$

(Where μ is the mean and σ is the standard deviation)

- **When to use it:** When your data follows a **Normal Distribution** (Bell Curve).
- When using algorithms like **Logistic Regression**, **SVM**, or **Principal Component Analysis (PCA)**.
- **The Upside:** It is much more robust to outliers than Min-Max scaling.

3. Robust Scaling (The "Anti-Outlier" Variant)

- If your data has **heavy outliers** that you don't want to delete, you use the RobustScaler. It uses the **Median** and **IQR** instead of Mean and Standard Deviation.

Python Side-by-Side Comparison

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler

# --- STEP 1: Create data with different scales and an outlier ---
data = {
    'Age': [20, 25, 30, 35, 40],            # Small range
    'Income': [40000, 50000, 60000, 70000, 1000000] # Large range + Outlier (1M)
}
df = pd.DataFrame(data)

# --- STEP 2: Initialize the Scalers ---
min_max = MinMaxScaler()
std_scaler = StandardScaler()
robust = RobustScaler()
```

```
# --- STEP 3: Apply and Compare ---
# Note: We fit_transform on the columns
df['MinMax_Income'] = min_max.fit_transform(df[['Income']])
df['Std_Income'] = std_scaler.fit_transform(df[['Income']])
df['Robust_Income'] = robust.fit_transform(df[['Income']])

print("--- Comparison of Scaling Methods ---")
print(df[['Income', 'MinMax_Income', 'Std_Income', 'Robust_Income']])

"""
UNDERSTANDING THE OUTPUT:
1. MinMax: Notice the first four incomes are squashed near 0 (0.00 to 0.03)
   because of the 1,000,000. It 'hides' the differences between 40k and 70k.

2. Standard: The mean is moved to 0. The 1,000,000 becomes a large positive number (approx 1.9).

3. Robust: Because it uses the median, the 1,000,000 doesn't 'pull' the scale
   as hard. The differences between the first four incomes remain visible.
"""
```

Deep-Dive Decision Matrix

Feature	Min-Max	Standardization	Robust Scaler
Output Range	Strictly [0, 1]	Not bounded (usually -3 to 3)	Not bounded
Outlier Sensitivity	Very High (Ruins the scale)	Medium	Low (Best for messy data)
Algorithm Choice	KNN, Neural Nets	Linear/Logistic Reg, PCA	Any model with many outliers
Distribution	Useful for non-Gaussian	Best for Gaussian (Bell Curve)	Any

A Quick Warning (Data Leakage Reminder!)

When you scale your data:

- Split into X_train and X_test.
- scaler.fit(X_train) <--- **ONLY learn from training data!**
- X_train_scaled = scaler.transform(X_train)
- X_test_scaled = scaler.transform(X_test)

Class Imbalance Handling

- This is for when you have 990 "Safe" transactions and only 10 "Fraud" transactions. How do you teach a model to find those 10?
- In many real-world scenarios (like fraud detection, rare disease diagnosis, or spam filtering), your data is **imbalanced**. This means one class (e.g., "Legit Transaction") far outnumbers the other (e.g., "Fraud").
- If 99% of your data is "Legit," a lazy model will simply predict "Legit" for every single case and achieve **99% accuracy**, while failing to catch even a single fraudster.

1. The Strategy: Re-sampling

To fix this, we balance the scales. There are two primary ways:

- **Oversampling (SMOTE):** Creating synthetic (fake but realistic) examples of the minority class.
  - *Analogy:* You have 10 "Fraud" photos and 990 "Legit" photos. You use AI to generate 980 more "Fraud" photos so the counts match.
- **Undersampling:** Randomly deleting examples from the majority class.
  - *Analogy:* You throw away 980 "Legit" photos and keep only 10, so you have 10 of each. (Warning: You lose a lot of information this way!)

2. Python Code: Handling Imbalance

We use the library imblearn (specifically the SMOTE - Synthetic Minority Over-sampling Technique) for this.

```
import pandas as pd
from sklearn.datasets import make_classification
from imblearn.over_sampling import SMOTE
from collections import Counter

# --- STEP 1: Create an imbalanced dataset ---
# We create 1000 rows where 95% are Class 0 and 5% are Class 1
X, y = make_classification(n_samples=1000, n_features=2, n_redundant=0,
                           n_clusters_per_class=1, weights=[0.95], flip_y=0, random_state=1)

print(f"Original class distribution: {Counter(y)}")
# Output: Counter({0: 950, 1: 50})

# --- STEP 2: Apply SMOTE (Oversampling) ---
# SMOTE looks at the '50' minority cases and creates '900' new synthetic ones
# by looking at the distance between the existing minority points.
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)
```

```
print(f"Resampled class distribution: {Counter(y_resampled)}")
# Output: Counter({0: 950, 1: 950})

"""
UNDERSTANDING THE PROCESS:
1. SMOTE doesn't just 'copy-paste' the minority rows.
2. It finds a minority point, looks at its 'neighbors', and draws a
   line between them to create a NEW point in the middle.
3. This helps the model learn the 'region' of the minority class
   rather than just memorizing specific rows.
"""
```

3. Alternative: Class Weights

- If you don't want to change your data, you can change the **Importance** inside the model. Most models in Scikit-learn (like Logistic Regression or Random Forest) have a parameter called ***class_weight='balanced'.***
- **How it works:** Every time the model makes a mistake on a "Fraud" case, it gets "punished" much harder than if it misses a "Legit" case. This forces the model to pay extra attention to the minority.

4. Important Evaluation Rule

When data is imbalanced, **Accuracy is a lie.**

You must use different metrics which we will cover in **Unit III**:

- **Precision:** How many of the "Fraud" alerts were actually fraud?
- **Recall:** Did we catch *all* the fraudsters?
- **F1-Score:** The balance between the two.

Congratulations! You have finished Unit I

We have covered:

- **Types of Data** (Nominal, Ordinal, Discrete, Continuous)
- **Data Pre-processing** (The workflow)
- **Data Leakage** (The "Cheating" concept)
- **Missing Values** (Imputation: Mean/Median/Mode)
- **Outliers** (IQR Method & Capping)
- **Categorical Data** (One-Hot, Ordinal, Label Encoding)
- **Scaling & Normalization** (Min-Max, Standard, Robust)
- **Class Imbalance** (SMOTE & Under-sampling)

UNIT II Contents

- Feature Engineering : variance threshold Feature Selection, correlation-based removal Feature Selection, forward selection, backward elimination, tree-based feature importance, Feature Extraction, creating new features, aggregation features
- Dimensionality Reduction : Curse of dimensionality,, Principal Component Analysis, explained variance ration, Linear Discriminant Analysis

Unit II: Feature Engineering & Dimensionality Reduction!

- If Unit I was about "cleaning the ingredients," Unit II is about "choosing the right ingredients" and "creating new flavors." A model is only as good as the features you give it.
- Let's start with **Feature Selection**. This is the process of dropping irrelevant or redundant features to make the model faster and more accurate.

1. Variance Threshold (The "Low Information" Filter)

This is the simplest form of feature selection. It looks at each column and asks: *"Does this data actually change?"*

- **The Concept:** If a column has the same value (zero variance) or almost the same value for every row, it provides no useful information.
- **Example:** If you are predicting house prices and you have a column "Is_on_Earth" where every row is "Yes," that feature is useless.
- **The Goal:** Remove features where the variance is below a certain threshold.

Python Code: Variance Threshold

We use Scikit-learn's VarianceThreshold.

```
import pandas as pd
from sklearn.feature_selection import VarianceThreshold

# --- STEP 1: Create a dataset with a 'Constant' column ---
data = {
    'Feature_A': [1, 2, 3, 4, 5],     # High variance (Good)
    'Feature_B': [1, 1.1, 1, 1, 1],   # Very low variance (Nearly constant)
    'Feature_C': [0, 0, 0, 0, 0]      # Zero variance (Useless)
}
df = pd.DataFrame(data)

# --- STEP 2: Initialize the selector ---
# Threshold=0.1 means: "Drop features where 90% or more values are the same"
selector = VarianceThreshold(threshold=0.1)

# --- STEP 3: Fit and transform ---
reduced_data = selector.fit_transform(df)
```

```
# Convert back to DataFrame to see which columns survived
selected_columns = df.columns[selector.get_support()]
df_reduced = pd.DataFrame(reduced_data, columns=selected_columns)

print("--- Original Data ---")
print(df)
print("\n--- After Variance Threshold (Feature_B and C removed) ---")
print(df_reduced)

"""
EXPLANATION:
1. Feature_C had 0 variance (all zeros), so it was immediately dropped.
2. Feature_B was so close to constant that it fell below our 0.1 threshold.
3. Only Feature_A was 'dynamic' enough to be kept for the model.
"""
```

2. Correlation-Based Feature Selection

- Sometimes two features are "Too Good Friends."
  If Feature_A and Feature_B are perfectly correlated (e.g., Temperature in Celsius and Temperature in Fahrenheit), you don't need both.
- **The Concept:** If two features are highly correlated (e.g., > 0.90), they provide redundant information. Removing one reduces "noise" and helps models like Linear Regression perform better.
- **The Tool:** We use the **Pandas.corr()** method.

Python Code: Correlation Removal

```
import pandas as pd
import numpy as np

# --- STEP 1: Create data with highly correlated columns ---
data = {
    'Height_cm': [160, 170, 180, 190],
    'Height_inches': [63, 67, 71, 75], # Perfectly correlated with cm
    'Weight_kg': [60, 70, 85, 95]
}
df = pd.DataFrame(data)

# --- STEP 2: Create a Correlation Matrix ---
corr_matrix = df.corr().abs()

# --- STEP 3: Identify columns to drop ---
# We look for correlations higher than 0.95
upper_triangle = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
to_drop = [column for column in upper_triangle.columns if any(upper_triangle[column] > 0.95)]
```

```
df_filtered = df.drop(columns=to_drop)

print(f"Columns to drop (too redundant): {to_drop}")
print("\n--- Filtered Data ---")
print(df_filtered)

"""
EXPLANATION:
'Height_cm' and 'Height_inches' tell the exact same story.
By dropping one, we make the model's job simpler without losing any real info.
"""
```

Part 1: The "Upper Triangle" Logic

The Scenario: You have a correlation table. It’s a square.

- The diagonal is always **1.0** (because Height is 100% correlated with Height).
- The top-right half is a **mirror image** of the bottom-left half (Height vs. Weight is the same as Weight vs. Height).
- If we don't "mask" half of the table, we will accidentally try to delete **both** columns because they are "highly correlated" with each other.

```
# 1. Create a matrix of True/False in a triangle shape
# np.triu stands for 'Triangle Upper'
mask = np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)

# 2. 'where' keeps only the top-right part of the table and hides the rest
upper_triangle = corr_matrix.where(mask)

# 3. This list comprehension says:
# 'Look at each column. If ANY value in that column is > 0.95, put that name in my drop list.'
to_drop = [column for column in upper_triangle.columns if any(upper_triangle[column] > 0.95)]
```

The Line: np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)

1. np.ones(corr_matrix.shape)

- **What it does:** Creates a table of the exact same size as your correlation matrix, but fills every single cell with the number
- **Example:** If your correlation matrix is 3×3, np.ones creates: $\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$

2. np.triu(..., k=1)

- **np.triu** stands for **Triangle Upper**. It takes a square and turns the bottom-left part into **zeros**.**k=1** is the "offset." If $k = 0$, it keeps the diagonal (the middle line).
- If $k = 1$, it also turns the diagonal into **zeros**, keeping only the values **above** the diagonal. $\begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$

3. .astype(bool)

- **What it does:** Changes the numbers into "True" or "False".
- In Python, **0 becomes False** and **1 becomes True**.
- **Final Stencil:** $\begin{bmatrix} False & True & True \\ False & False & True \\ False & False & False \end{bmatrix}$
- **Why did we do all that? (The Real-World Logic)**
- Imagine you have a spreadsheet of correlations between **Height**, **Weight**, and **Age**.

	Height	Weight	Age
Height	1.0	**0.8**	0.2
Weight	**0.8**	1.0	0.3
Age	0.2	0.3	1.0

Explanation

- Notice that **0.8** appears twice (Height-Weight and Weight-Height). If we ask Python "Which columns have a correlation > 0.7?", it will say "Both Height and Weight!" and it might delete **both**, leaving you with no data!
- By using our **True/False Stencil**, we tell Python: *"Only look at the cells where the stencil is **True**."*
- It ignores the diagonal (1.0) because we don't care that Height correlates with Height.
- It ignores the bottom-left because it's just a repeat.
- It only sees the **0.8** once. Now it knows: "Okay, I only need to drop **one** of these two."

Deep-Dive Question:

- Imagine you have two columns: **"Years of Experience"** and **"Salary"**. They will likely have a very high correlation.
- **Should you drop one of them using the correlation method?**

****(Think about which one is likely the input and which is the target!)*

1. Feature vs. Target

- In Machine Learning, we usually use one to predict the other.
- **Feature (X):** Years of Experience
- **Target (y):** Salary

- Correlation-based removal is only for **between two Features (X_1 and X_2)**.
- We never drop a feature just because it is highly correlated with the **Target (y)**.
- In fact, a high correlation with the target is **exactly what we want**—it means that feature is a very strong predictor!

2. Causality

- Even if they were both features (e.g., you are trying to predict "Job Satisfaction"), you only drop one if they are **redundant** (like $Height_cm$ and $Height_inches$).
- Experience and Salary are related, but they aren't the same thing. One is a measure of time, the other is a measure of money.

3. Forward Selection & Backward Elimination (Wrapper Methods)

- These are called **Wrapper Methods** because they "wrap" a machine learning model around the selection process.

A. Forward Selection

- Start with **zero features**.
- Test every single feature one by one.
- Keep the one that gives the best model performance.
- Now, try adding a second feature to the first one. Keep the best pair.
- Repeat until adding new features no longer improves the score.

B. Backward Elimination

- Start with **all features**.
- Remove the "weakest" feature (the one that contributes the least to the model).
- Check the performance. If it didn't get worse, keep it removed.
- Repeat until you are left with only the most impactful features.

Python Code: Sequential Feature Selection

We can use Scikit-learn's SequentialFeatureSelector to automate this.

```
import pandas as pd
from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris

# --- STEP 1: Load a dataset (Iris has 4 features) ---
data = load_iris()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

# --- STEP 2: Initialize a Model ---
# The selector needs a model to 'test' the features
lr = LogisticRegression(max_iter=200)

# --- STEP 3: Initialize Forward Selector ---
# n_features_to_select='auto' or a specific number like 2
sfs = SequentialFeatureSelector(lr, n_features_to_select=2, direction='forward')
```

```
# --- STEP 4: Fit the selector ---
sfs.fit(X, y)

# --- STEP 5: See which features were chosen ---
selected_features = X.columns[sfs.get_support()]

print(f"Original Features: {list(X.columns)}")
print(f"Selected Features (Forward): {list(selected_features)}")

"""
EXPLANATION:
- We started with 4 features (sepal length, sepal width, petal length, petal width).
- The Forward Selector tried combinations and found that just using 2 specific features (usually the petal measurements) was enough to get a great result.
- This makes our model simpler and prevents 'Overfitting'.
"""
```

Part 2: Sequential Feature Selection

This code is like a **"Talent Scout"** trying to find the 2 best players for a team.

```
# --- THE IMPORTS ---
import pandas as pd  # To handle our data tables
from sklearn.feature_selection import SequentialFeatureSelector # The 'Talent Scout'
from sklearn.linear_model import LogisticRegression # The 'Coach' who evaluates players
from sklearn.datasets import load_iris # A famous dataset of flowers

# --- STEP 1: Load Data ---
data = load_iris()
# X = The 4 features of the flower (Sepal length/width, Petal length/width)
X = pd.DataFrame(data.data, columns=data.feature_names)
# y = The type of flower (the answer we want to predict)
y = data.target

# --- STEP 2: Initialize the 'Coach' (Model) ---
# The Scout needs a Coach to tell them if a feature is good or not.
# max_iter=200 just gives the coach more time to think/calculate.
lr = LogisticRegression(max_iter=200)
```

```
# --- STEP 3: Initialize the 'Talent Scout' (Selector) ---
# lr: The coach who will test the features.
# n_features_to_select=2: We only want the top 2 players.
# direction='forward': Start with 0 features, and add the best one at a time.
sfs = SequentialFeatureSelector(lr, n_features_to_select=2, direction='forward')

# --- STEP 4: The Selection Process ---
# The scout now runs a tournament.
# It tries Feature 1, then Feature 2... then it tries (1 and 2), (1 and 3), etc.
sfs.fit(X, y)

# --- STEP 5: Getting the Results ---
# sfs.get_support() returns a list like [False, False, True, True]
# This tells us which columns 'won' the tournament.
selected_features = X.columns[sfs.get_support()]

# --- PRINTING ---
print(f"Original Features: {list(X.columns)}")
print(f"Selected Features: {list(selected_features)}")
```

Real World Example:

- Imagine you are building a "Weather App." You have 4 sensors: **Temperature, Humidity, Wind Speed, and Sensor Serial Number.**
- The Forward Selector tries each one.
- It realizes "Temperature" is the best.
- Then it tries "Temperature + Humidity" and sees the accuracy goes up.
- Then it tries "Temperature + Humidity + Serial Number" and realizes the accuracy **doesn't** change because the Serial Number is useless.
- It stops and tells you: **"Just use Temperature and Humidity."**

4. Tree-based Feature Importance (The "Decision-Maker" Method)

- This is a very popular "shortcut." If you use a model like a **Random Forest** or **Decision Tree**, the model automatically calculates which features it used most often to make its decisions.
- **The Concept:** When a **Decision Tree** (like a flowchart) looks at data, it has to pick a "Starting Question." It usually picks the most important feature first. We can ask the model to give us a "Score" (from 0 to 1) for every column based on how much it helped the model make a decision

Python Code: Tree Importance

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier # A team of Decision Trees
from sklearn.datasets import load_iris

# --- STEP 1: Load Data ---
data = load_iris()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

# --- STEP 2: Initialize and Train the Model ---
# n_estimators=100 means 'Use a team of 100 small trees'
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X, y)

# --- STEP 3: Get the Importance Scores ---
# The model has a built-in property called 'feature_importances_'
importances = model.feature_importances_
```

```
# --- STEP 4: Put them in a pretty table ---
feat_importances = pd.DataFrame(importances, index=X.columns,
columns=["Importance"])
# Sort them so the most important is at the top
feat_importances = feat_importances.sort_values(by='Importance',
ascending=False)

print("--- Feature Importance Scores ---")
print(feat_importances)

"""
EXPLANATION:
- 'RandomForestClassifier': This is a 'Group of Experts' model.
- 'model.fit(X, y)': The experts look at the flower data and learn the patterns.
- 'model.feature_importances_': The experts report back: 'Hey, we used Petal Width
  way more than Sepal Length to solve this!'
- 'sort_values': We put the 'MVPs' (Most Valuable Players) at the top of the list.
"""
```

Real World Example:

You want to predict if a movie will be a hit. You have: **Budget, Actor Name, Release Date, and Director's Shoe Size.**

- The Random Forest will run.
- It will see that "Director's Shoe Size" never helps it predict correctly. It gives it a score of **0.01**.
- It sees "Budget" helps a lot. It gives it **0.70**.
- Now you know exactly which features to keep!

Python Code: Visualising Feature Importance

```
import pandas as pd
import matplotlib.pyplot as plt # The standard plotting tool
import seaborn as sns          # A 'stylist' for our charts
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris

# --- STEP 1: Get the data and train the model (Same as before) ---
data = load_iris()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X, y)

# --- STEP 2: Prepare the data for the chart ---
# We create a DataFrame containing the feature names and their scores
feat_importances = pd.Series(model.feature_importances_, index=X.columns)

# We sort them so the chart looks organized (highest on top)
feat_importances = feat_importances.sort_values(ascending=True)

# --- STEP 3: Create the Plot ---
# 1. Set the size of the 'canvas'
plt.figure(figsize=(10, 6))
```

```
# 2. Use Seaborn to create a horizontal bar plot (barplot)
# palette='viridis' gives it a nice color gradient
sns.barplot(x=feat_importances.values, y=feat_importances.index, palette='viridis')

# 3. Add Labels (The 'Context')
plt.title('Which Features are the most important?')
plt.xlabel('Importance Score (0 to 1)')
plt.ylabel('Feature Name')

# 4. Show the plot
plt.show()

"""
EXPLANATION:
- 'plt.figure': Like buying a 10x6 inch frame for your picture.
- 'sns.barplot': This draws the actual bars. We put the 'Scores' on the X-axis
  and 'Names' on the Y-axis.
- 'sort_values': We sort them so the longest bar is at the top. This makes it
  instantly clear which feature is the 'winner'.
- 'plt.show()': This tells Python 'Okay, I'm done drawing, show me the window!'
```

What does this chart tell us?

- If you run this code, you will see that **Petal Width** and **Petal Length** have very long bars, while the "Sepal" features have very short ones.
- **The Insight:** If you were building a mobile app to identify flowers, you could save money/time by only asking users to measure the petals, because the sepals don't really help the model differentiate between the types.

Feature Extraction (Creating New Features)

- Selection was about *deleting* columns. **Extraction** is about *creating* new columns from the ones you already have. This is where your creativity comes in!

1. Creating New Features (Domain Knowledge)

- Sometimes, the relationship between two columns is more important than the columns themselves.
- **Example:** You have Weight and Height. A model might struggle with them separately. But if you create BMI = Weight / (Height^2), the model suddenly has a much more powerful predictor.

2. Aggregation Features

- This is common when you have "transactional" data.
- **Example:** You have a list of all purchases a customer made.
  - Total_Spent (Sum)
  - Average_Transaction_Value (Mean)
  - Purchase_Frequency (Count)
    These "Aggregated" numbers represent the customer much better than a list of individual receipts.

- Creating new features using math in **Pandas** is often called **Feature Construction**. This is where you use your "human intuition" to help the "machine" see patterns more clearly.
- In Pandas, we do this using **Vectorized Operations**. This is a fancy way of saying we can perform math on the entire column (all rows at once) without writing a "loop."

1. Simple Math-Based Features

Let's use a real-world example of an **E-commerce Store**. We have the "Unit Price" and the "Quantity," but we want the "Total Cost."

```
import pandas as pd


# --- STEP 1: Create a simple Sales dataset ---
data = {
    'Product': ['Laptop', 'Mouse', 'Monitor', 'Keyboard'],
    'Unit_Price': [1000, 25, 200, 50],
    'Quantity': [2, 10, 3, 5],
    'Weight_kg': [2.5, 0.1, 5.0, 1.2]
}
df = pd.DataFrame(data)
```

```
# --- STEP 2: Mathematical Creation ---
# We create a 'Total_Value' column by multiplying two existing columns
df['Total_Value'] = df['Unit_Price'] * df['Quantity']

# We can also do more complex math, like 'Value per Kilogram'
df['Value_per_kg'] = df['Total_Value'] / df['Weight_kg']

print("--- New Math-Based Features ---")
print(df)

"""
EXPLANATION:
- df['New_Column'] = ... : This is how you tell Pandas to add a new column.
- The math happens row-by-row automatically.
- For the first row: 1000 * 2 = 2000.
"""
```

2. Aggregation Features (The "Group By" Method)

- This is very common when you have multiple rows for the same person or category, and you want to summarize them.
- **Example:** You have a list of students and their scores in different subjects. You want to create a feature called **"Student_Average."**

```
# --- STEP 1: Create a dataset of Exam Scores ---
scores_data = {
    'Student_ID': [101, 101, 102, 102, 103],
    'Subject': ['Math', 'Science', 'Math', 'Science', 'Math'],
    'Score': [90, 80, 70, 60, 100]
}
df_scores = pd.DataFrame(scores_data)

# --- STEP 2: Aggregate using 'groupby' ---
# We group by Student_ID and calculate the mean score for each student
student_averages = df_scores.groupby('Student_ID')['Score'].mean().reset_index()
```

```
# Rename the column so it makes sense
student_averages.columns = ['Student_ID', 'Average_Score']

# --- STEP 3: Merge it back to the original data (Optional) ---
# This gives every row the context of how that student performs overall
df_final = pd.merge(df_scores, student_averages, on='Student_ID')

print("\n--- Aggregated Features ---")
print(df_final)

"""
EXPLANATION:
- 'groupby('Student_ID')': Put all rows belonging to the same student into a group.
- '['Score'].mean()': Calculate the average score for that specific group.
- 'pd.merge': Attach the calculated average back to the original table so
  the model can compare a single test score to the student's overall average.
```

3. Log Transformations (Reviewing Unit I)

- Sometimes "creating" a feature means changing its scale to make it more "normal" (like we did with the House Price example earlier).
- df['Log_Income'] = np.log1p(df['Income’])

- **Why do we do this? (The "In-Depth" Reason)**
- Machine Learning models (especially simpler ones like Linear Regression) are not very good at "discovering" division or multiplication on their own.
- If a house price is determined by **Price per Square Foot**, the model will learn much faster if you provide a Price_per_SqFt column directly, rather than making the model try to figure out the relationship between Total_Price and Area.

Dimensionality Reduction (PCA)

- We have learned how to **select** features (delete them) and **extract** features (create them). But what if you have **1,000 features** and you need to squash them down into just **10 features** without losing the "essence" of the data?
- This is where **Principal Component Analysis (PCA)** comes in. It’s like "compressing" a large file into a ZIP folder.
- Imagine you are trying to describe a person. You could list 1,000 things: their height, weight, hair length, eye colour, every single freckle location, etc. That is a lot of data!
- **Dimensionality Reduction** is the art of "compressing" those 1,000 features into 5 "Principal" traits (like "Physical Build" or "Facial Structure") that still capture the essence of who that person is.

1. The "Curse of Dimensionality"

Why do we want fewer features?

- **Speed:** More columns = more math = slower training.
- **Overfitting:** With too many columns, the model starts memorizing random noise instead of real patterns.
- **Visualisation:** Humans can't visualize 10-dimensional data. We need to squash it to 2D or 3D to see it on a screen.

2. Principal Component Analysis (PCA)

PCA is the most famous tool for this. It doesn't just "pick" columns; it **combines** them.

- **How it works:** It looks for the direction where the data varies the most. It creates new, artificial features called **Principal Components (PCs)**.
- **PC1** captures the most information.
- **PC2** captures the second most, and so on.

PCA (Continue...)

- PCA is **Unsupervised**. It does not care about the labels (e.g., whether a flower is a Rose or a Tulip). It only cares about **Variance** (spread).
- The "Camera Angle" Analogy
- Imagine a 3D object, like a **football**.
- If you take a photo from the side, you see a circle (2D).
- If you take a photo from the top, you see a circle (2D).
- PCA is like finding the **perfect camera angle** that captures the most "shape" of the object in a flat 2D photo.

- **Real-World Example: Facial Recognition (Eigenfaces)**
- In early facial recognition, instead of storing every pixel of a 1000x1000 image (1,000,000 features), PCA was used to find the "Principal Components" of faces. It turned out you only needed about **50 components** (eyes, nose shape, jawline) to identify a person.

- **Python: How many components do we need?**
- We use the **explained_variance_ratio_** to decide. Usually, we want to keep enough components to cover **95%** of the original information.

How it works?

- It finds new, uncorrelated axes called **Principal Components (PCs)**.
  - **PC1** is the direction where the data has the most spread (highest variance).
  - **PC2** is perpendicular (orthogonal) to PC1 and captures the next highest variance.
- **Mathematical Core**: It uses the **Covariance Matrix** of your data to find **Eigenvectors**(the directions of the PCs) and **Eigenvalues** (the "strength" or amount of variance in those directions).
- **Requirement**: You **must** scale/standardize your data (Mean=0, SD=1) before PCA, or features with large numbers will unfairly dominate the result.

- **Definition:** An **unsupervised** linear transformation technique used to compress data by identifying patterns based on **variance**.
- **Standardization:** Features are scaled so they have a mean of 0 and a variance of 1.
- **Covariance Matrix:** PCA calculates how each variable relates to others.
- **Eigen-Decomposition:**
  - **Eigenvectors:** These are the **directions** (axes) of the new space.
  - **Eigenvalues:** These are scalars that tell us **how much variance** is captured in each direction.
- **Explained Variance Ratio:** We rank eigenvectors by their eigenvalues. The top ones (PC1, PC2) represent the most "information."

Explained Variance Ratio

- This is a critical metric to decide how many components to keep.
- **Definition**: It tells you the percentage of total variance captured by each principal component.
- **Selection**: If PC1 explains 70% and PC2 explains 20%, keeping just those two captures 90% of your data's "information" while potentially throwing away dozens of other redundant features.
- **Scree Plot**: A common bar chart used to visualize these ratios to find the "elbow" point where adding more components gives diminishing returns.

3. Python Code: PCA

We will take the **Iris dataset** (4 features) and "compress" it into just **2 features** so we can plot it on a 2D graph.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris

# --- STEP 1: Load and Scale Data ---
# PCA is very sensitive to scale, so we MUST standardize first!
data = load_iris()
X = data.data
y = data.target

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# --- STEP 2: Initialize PCA ---
# n_components=2 means 'Squash my 4 columns into 2 new ones'
pca = PCA(n_components=2)

# --- STEP 3: Fit and Transform ---
X_pca = pca.fit_transform(X_scaled)
```

```
# Convert to a DataFrame for easy viewing
df_pca = pd.DataFrame(data=X_pca, columns=['PC1', 'PC2'])

# --- STEP 4: Check "Explained Variance Ratio" ---
# This tells us how much of the original 'information' we kept.
print(f"Information kept in PC1: {pca.explained_variance_ratio_[0]*100:.2f}%")
print(f"Information kept in PC2: {pca.explained_variance_ratio_[1]*100:.2f}%")

# --- STEP 5: Visualise the "Compressed" Data ---
plt.figure(figsize=(8, 6))
plt.scatter(df_pca['PC1'], df_pca['PC2'], c=y, cmap='viridis')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('Iris Data Compressed to 2D')
plt.show()

"""
EXPLANATION:
- 'StandardScaler': PCA works by measuring distances. If one column is 0-100  and another is 0-1, PCA will get confused. Scaling makes them equal.
- 'PCA(n_components=2)': This is the 'Compressor' setting.
- 'explained_variance_ratio_': This is like the 'File Size' check. If these two  numbers add up to 95%, it means we kept 95% of the logic but used 50% less space!
- 'plt.scatter': Since we now only have 2 columns (PC1 and PC2), we can finally draw the data on a normal flat graph.
```

PCA CODE 2

```
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_breast_cancer

# --- STEP 1: Load complex data (30 features) ---
data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)

# --- STEP 2: Standardize (MANDATORY for PCA) ---
# If one feature is 1000 and another is 0.1, PCA thinks the 1000 is
# 'more variant' just because the number is bigger. Scaling fixes this.
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
# --- STEP 3: Run PCA with NO limit ---
# We want to see how much info is in EVERY possible component
pca = PCA()
pca.fit(X_scaled)

# --- STEP 4: Calculate Cumulative Variance ---
# This adds up the info: PC1 + PC2 + PC3...
cumulative_variance = np.cumsum(pca.explained_variance_ratio_)

# --- STEP 5: Find how many components give us 95% info ---
n_95 = np.argmax(cumulative_variance >= 0.95) + 1

print(f"Original Features: {X.shape[1]}")
print(f"Features needed for 95% info: {n_95}")

"""
EXPLANATION:
- We started with 30 columns.
- PCA shows that we can represent 95% of the 'logic' of this  medical data using only about 10 new 'Principal' columns.
- We just reduced the work for our future model by 66%!
```

Knowledge Check (Quiz)

- **Which technique is better for visualizing clusters when labels are unknown?**
- **True/False:** PCA reduces the number of features by selecting the most important original columns.
- **What does a "Scree Plot" represent?**

4. Linear Discriminant Analysis (LDA)

- LDA is similar to PCA but with a **major difference**:
- **PCA (Unsupervised):** It ignores the "Target" (y). It just looks for the most "spread out" data.
- **LDA (Supervised):** It looks at the "Target" (y). It tries to squash the data in a way that keeps the different classes (e.g., different flowers) as far apart as possible.
- **The "Separator" Philosophy**
- While PCA tries to find the most "spread," LDA tries to find the axis that **maximizes the distance between classes** and **minimizes the spread within each class.**
- **PCA Goal:** Find the most "interesting" view of the data overall.
- **LDA Goal:** Find the view that makes the groups look as different as possible.

- LDA is **Supervised**. It **DOES** care about the labels (y).

Real-World Example: Customer Loyalty

- Imagine you have data on customers (Age, Income, Spend).
- **PCA** would tell you how customers vary in general.
- **LDA** would find the specific combination of Age and Income that **best separates**"Loyal Customers" from "Churning Customers."

Python: PCA vs LDA

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

# --- STEP 1: Initialize LDA ---
# Note: LDA needs 'n_classes - 1' components.
# Since Iris has 3 types of flowers, LDA can only produce 2 components.
lda = LinearDiscriminantAnalysis(n_components=2)

# --- STEP 2: Fit and Transform ---
# CRITICAL: Unlike PCA, LDA needs the 'y' (target) during fitting!
X_lda = lda.fit_transform(X_scaled, data.target)

print(f"Original Shape: {X_scaled.shape}")
print(f"LDA Shape: {X_lda.shape}")
"""
EXPLANATION:
- 'lda.fit(X, y)': LDA looks at the features (X) AND the flower types (y).
- It calculates: 'How can I squash these 4 features into 2 so that the Setosa, Versicolor, and Virginica flowers are in totally separate piles?
```

3. Comparison Table: Which one to use?

Feature	PCA	LDA
Type	Unsupervised	Supervised
Focus	Maximizing Variance (Spread)	Maximizing Class Separability
	No	Yes
Main Use Case	Compression & Visualization	Feature Extraction for Classification
Limit	Can have as many PCs as features	Max components = (Number of Classes - 1)

Deep-Dive Question:

- If you are working on a **Digit Recognition** project (recognizing handwritten numbers 0-9), and your data is very noisy:
- If you want to **clean the noise** without caring about the labels, which would you use?
- If you want to find the features that **best distinguish** a "1" from a "7," which would you use?
- **Would you like to see a "Decision Plot" that compares how PCA and LDA group the same data differently?**

Summary of Unit II

Method	Goal	Technique
Selection	Keep the best real columns	Variance, Correlation, Forward/Backward
Extraction	Create better columns	Math ($A \times B$), Aggregations (Mean)
Reduction	Compress all columns	PCA (Math-based compression)

UNIT - III

- Linear & Probabilistic Classification Models : Perceptron, Logistic Regression, Naïve Bayes Classifier, Model evaluation using confusion matrix, precision, recall, F1-Score, Introduction to supervised Learning, Nonlinear & Distance-Based Models : K-Nearest Neighbors, Decision Tree, Support Vector, Machine, Model evaluation using ROC Curve, ROC-AUC, Precision-Recall Curve, AUC-PR

Unit III - Perceptron & Logistic Regression

1. The Concept: What is Classification?

- **The Real-World Analogy: The "Bank Loan" Filter**
- Imagine a bank clerk. Every time someone applies for a loan, the clerk looks at two things: **Credit Score** and **Annual Income**.
- The clerk has a "mental line."
- If a customer falls above that line, they are **Approved (Class 1)**.
- If they fall below, they are **Rejected (Class 0)**.
- Classification is simply the machine finding that **"boundary line"** to separate different groups.

2. The Perceptron: The "Original" Classifier

- **Definition:** The simplest form of a Neural Network. It is a **Binary Classifier** that maps inputs to a single output (0 or 1) using a step function.
- **In-Depth Explanation: How it "Thinks"**
- **Weights (w):** Each feature (like Income) is assigned a weight. If Income is very important for the loan, its weight will be high.
- **Bias (b):** A threshold value that allows the model to shift the decision boundary.
- **Linear Sum (z):** $z = (x_1 \cdot w_1) + (x_2 \cdot w_2) + b$

Activation (Step Function):

- If $z > 0 \rightarrow$ Predict **1**
- If $z \leq 0 \rightarrow$ Predict **0**

Deep Dive Question: Can a Perceptron solve the "XOR" problem?

- *Answer:* No. A Perceptron can only draw a **straight line**. If the data is mixed (non-linear), the Perceptron fails. This is its biggest limitation.

1. The Perceptron: The Biological Neuron’s Digital Twin

- The Perceptron is the simplest form of a neural network. It is a **linear binary classifier**.
- **The Mechanism:**
  It takes input features (x), multiplies them by weights (w), adds a bias (b), and passes the result through a **Step Function**.

- $z = \sum(w_i \cdot x_i) + b$
- $\hat{y} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$
- **The Learning Rule (Weight Update):**
  Unlike complex models, the Perceptron updates weights only when it makes a mistake:
- $w_{new} = w_{old} + \eta \cdot (y - \hat{y}) \cdot x$
- where
- η is the learning rate. If the prediction is correct,
- $(y - \hat{y})$ is zero, and no change happens.

- **The Fatal Flaw:** The Perceptron only converges if the data is **linearly separable**. If you try to teach it an XOR gate (where you can't draw a single straight line to separate classes), it will loop forever.

Real-World Analogy

- Imagine a **Credit Card Approval** process.
- **Inputs:** Income, Credit Score, Debt.
- **Weights:** The bank decides Credit Score is 3x more important than Income.
- **Threshold:** If the weighted sum is
- > 700, "Approve" (1), otherwise "Reject" (0).
- **Learning:** If the bank rejects a person who would have been a good payer, they "adjust their weights" to be slightly more lenient next time.

```
# 2. Start the 'Learning' loop
    for _ in range(self.epochs):
        # idx: index of the row, x_i: the actual data row
        for idx, x_i in enumerate(X):

            # 3. Calculate the RAW SCORE (z = w1*x1 + w2*x2 + ... + bias)
            # np.dot is a shortcut for multiplying and adding everything
            linear_output = np.dot(x_i, self.weights) + self.bias

            # 4. Use the STEP FUNCTION (The Activation)
            # If score >= 0, predict 1 (Positive). If < 0, predict 0 (Negative).
            y_predicted = 1 if linear_output >= 0 else 0

            # 5. THE LEARNING RULE (Wait, did I get it wrong?)
            # Update = LearningRate * (Actual - Predicted)
            # If Actual is 1 and Predicted is 0, Update is Positive (Increase weights)
            # If Actual is 0 and Predicted is 1, Update is Negative (Decrease weights)
            # If they match, Update is 0 (No change!)
            update = self.lr * (y[idx] - y_predicted)
```

Python Implementation - Perceptron

```
import numpy as np  # A library for fast math (Linear Algebra)
import matplotlib.pyplot as plt  # For drawing graphs (optional)


class PerceptronCustom:
    def __init__(self, learning_rate=0.01, epochs=1000):
        """
        This is the "Setup" of our brain. learning_rate (eta): How big a step we take when we fix a mistake. epochs: How many times the brain looks at the entire dataset to learn.
        """
        self.lr = learning_rate
        self.epochs = epochs
        self.weights = None  # The "Importance" given to each feature
        self.bias = 0        # The "Threshold" to shift the decision line


    def fit(self, X, y):
        """
        The 'Training' phase.
        X: The input data (like Age, Income)
        y: The correct answers (0 or 1)
        """
        n_samples, n_features = X.shape # n_samples = rows, n_features = columns
        # 1. Start with zero knowledge: Every weight is set to 0 initially
        self.weights = np.zeros(n_features)
        self.bias = 0
```

```
# 6. Apply the correction to weights and bias
            self.weights += update * x_i
            self.bias += update

    def predict(self, X):
        """
        The 'Testing' phase. Use the learned weights on new data.
        """
        linear_output = np.dot(X, self.weights) + self.bias
        # Return 1 if score >=0, else return 0
        return np.where(linear_output >= 0, 1, 0)
```

The **Perceptron** is basically a **"Mistake-Learning Machine."**

- You are a **Bouncer** at a club. Your job is to decide: **Let them in (1)** or **Kick them out (0)**.
  You have two rules:
- Are they wearing nice shoes? (x_1)
- Are they acting rude? (x_2)

- At first, you have **no idea** how to do your job. You are a "Fresh Bouncer."
- **Step 1 (The Score):** A guy arrives. You give a "score" to his shoes and his behavior.
- **Step 2 (The Decision):** Because you're new, you just guess. You let him in.
- **Step 3 (The Mistake):** The guy starts a fight inside. Your boss (the **Labels/Target**) shouts: *"Hey! You shouldn't have let him in!"*
- **Step 4 (The Update):** You think, *"Okay, I need to be stricter about rude behavior."* You change your internal "Importance" (**Weights**) for the next person.

Let’s look at the "Hard Parts" of the code again, but with "Bouncer" language:

- **self.weights**: This is how much you care about shoes vs. behavior.
- **self.bias**: This is how "strict" or "chill" you are naturally.
- **np.dot(x_i, self.weights)**: This is just you calculating the "Total Score" for the person standing in front of you.
- **y[idx] - y_predicted**: This is your boss telling you the error.
  - If Boss says 0 (Kick out) and you said 0 (Kick out) → **0 error**. (Good job!)
  - If Boss says 0 (Kick out) and you said 1 (Let in) → **-1 error**. (You were too nice, adjust!)

Why "Linearly Separable" matters:

- Imagine if the club rule was: *"Let them in ONLY if they have nice shoes OR if they are polite, BUT NOT BOTH."* (This is the famous **XOR** problem).
  A bouncer who only uses a straight line (a simple rule) will get confused and can never learn this rule perfectly.

Knowledge Check (The "Vibe" Check)

3. Logistic Regression: The "Probability" Expert

- **Definition:** Despite the name "Regression," it is used for **Classification**. Instead of a hard "Yes/No," it calculates the **Probability** (0% to 100%) of a class.
- **The Real-World Analogy: The "Weather Forecast"**
- A Perceptron says: "It will rain." (Hard 0 or 1).
  Logistic Regression says: "There is an 85% chance it will rain." (Probability).
- **The Sigmoid Function (The Secret Sauce)**
- To turn any number into a probability between 0 and 1, we use the **Sigmoid Function**:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$